

Connecting to HubSpot API with R

Introduction

This document demonstrates how to connect to the HubSpot API using R, brought to you by the Slapshot Rabbit! We'll cover authentication setup, basic API calls, and data retrieval examples.



Setup and Authentication

First, let's load the required packages:

```
library(httr2)
library(jsonlite)
library(dplyr)
library(tibble)
library(knitr)
library(purrr)

# Optional: for data visualization
library(ggplot2)
library(scales)
```

HubSpot API Authentication

HubSpot uses API keys or OAuth for authentication. For this example, we'll use a Private App access token, which is the recommended approach for server-to-server integrations.

Step 1: Create a Private App in HubSpot

1. Go to your HubSpot account settings
2. Navigate to **Integrations > Private Apps**
3. Click **Create a private app**
4. Configure the required scopes (e.g., **contacts**, **companies**, **deals**)
5. Generate and copy your access token

Step 2: Store Your API Key Securely

Important: Never hardcode API keys in your code. Use environment variables instead.

Create a `.env` file in your project root (add this to `.gitignore`):

```
HUBSPOT_ACCESS_TOKEN=your_access_token_here
```

Or set the environment variable in your R session:

```
# Option 1: Set environment variable in R
# Sys.setenv(HUBSPOT_ACCESS_TOKEN = "your_access_token_here")

# Option 2: Use .Renviron file
# Add this line to your .Renviron file:
# HUBSPOT_ACCESS_TOKEN=your_access_token_here
```

```
# Load the access token from environment
hubspot_token <- Sys.getenv("HUBSPOT_ACCESS_TOKEN")

if (hubspot_token == "") {
  stop("HubSpot access token not found. Please set HUBSPOT_ACCESS_TOKEN environment variable")
}
```

Helper Functions

Let's create some helper functions for making API calls:

```
# Base URL for HubSpot API
HUBSPOT_BASE_URL <- "https://api.hubapi.com"

# Helper function to make authenticated requests to HubSpot API
hubspot_request <- function(endpoint, method = "GET", body = NULL) {
  url <- paste0(HUBSPOT_BASE_URL, endpoint)

  req <- request(url) |>
    req_headers(
      Authorization = paste("Bearer", hubspot_token),
      `Content-Type` = "application/json"
    )

  if (method == "POST" && !is.null(body)) {
    req <- req |> req_body_json(body)
  }

  if (method == "GET") {
    response <- req |> req_perform()
  } else if (method == "POST") {
    response <- req |> req_method("POST") |> req_perform()
  }

  # Parse JSON response
  content <- response |> resp_body_json()
  return(content)
}

# Function to get all results with pagination
hubspot_get_all <- function(endpoint, limit = 100) {
  all_results <- list()
  after <- NULL
  page <- 1

  repeat {
    # Build query parameters
    query_params <- list(limit = limit)
    if (!is.null(after)) {
      query_params$after <- after
    }
  }
}
```

```

}

# Construct URL with query parameters
query_string <- paste(names(query_params), query_params, sep = "=", collapse = "&")
full_endpoint <- paste0(endpoint, "?", query_string)

cat("Fetching page", page, "...\\n")

response <- hubspot_request(full_endpoint)

# Add results to our list
if (!is.null(response$results)) {
  all_results <- c(all_results, response$results)
}

# Check if there are more pages
if (is.null(response$paging) || is.null(response$paging$`next`)) {
  break
}

after <- response$paging$`next`$after
page <- page + 1
}

return(all_results)
}

```

Testing the Connection

Let's test our connection by fetching account information:

```

# Test the connection by getting account info
tryCatch({
  account_info <- hubspot_request("/integrations/v1/me")

  cat("Connection successful!\\n")
  cat("Portal ID:", account_info$portalId, "\\n")
  cat("App ID:", account_info$appId, "\\n")
  cat("Hub ID:", account_info$hubId, "\\n")
  cat("Scopes:", paste(account_info$scopes, collapse = ", "), "\\n")
}, error = function(e) {

```

```
    cat("Connection failed:", e$message, "\n")
  })
```

Connection successful!
Portal ID: 7499862
App ID:
Hub ID:
Scopes:

Working with Contacts

Fetch Contacts

```
# Fetch contacts with specific properties
contacts_endpoint <- "/crm/v3/objects/contacts"

# Specify which properties to retrieve
properties <- c("firstname", "lastname", "email", "company", "phone", "createdate")
properties_param <- paste(properties, collapse = ",")

contacts_url <- paste0(contacts_endpoint, "?properties=", properties_param)

# Get first 10 contacts
contacts_response <- hubspot_request(contacts_url)

# Convert to data frame
contacts_df <- map_dfr(contacts_response$results, function(contact) {
  props <- contact$properties
  tibble(
    id = contact$id,
    firstname = props$firstname %||% NA,
    lastname = props$lastname %||% NA,
    email = props$email %||% NA,
    company = props$company %||% NA,
    phone = props$phone %||% NA,
    createdate = props$createdate %||% NA
  )
})
```

```
# Display first few contacts
kable(head(contacts_df), caption = "Sample HubSpot Contacts")
```

Table 1: Sample HubSpot Contacts

id	firstname	lastname	email	company	phone	createdate
101	Terry	Ekkamolkul	terry.ekkamolkul@t-mobile.com	T-Mobile	NA	2020-04-15T04:50:51.824Z
151	Nichole	Monhait	nichole@posit.co	Posit	224-213-0463	2020-05-19T23:26:57.898Z
201	Rozenia	Cummings	cummings.rozenia@gene.com	NA	NA	2020-05-20T16:49:57.834Z
251	Kazi	Mustaqhusain	kazi.mustaqhusain@gene.com	NA	NA	2020-05-20T16:54:41.900Z
401	Neeraj	Agarwal	agarwal.neeraj@gene.com	Genentech	NA	2020-05-20T18:51:55.451Z
451	Alpna	Kak	kak.alpna@gene.com	NA	NA	2020-05-20T18:54:56.597Z

Contact Statistics

```
if (nrow(contacts_df) > 0) {
  cat("Total contacts retrieved:", nrow(contacts_df), "\n")
  cat("Contacts with email:", sum(!is.na(contacts_df$email)), "\n")
  cat("Contacts with phone:", sum(!is.na(contacts_df$phone)), "\n")
  cat("Contacts with company:", sum(!is.na(contacts_df$company)), "\n")
}
```

```
Total contacts retrieved: 10
Contacts with email: 10
Contacts with phone: 2
Contacts with company: 4
```

Working with Companies

Fetch Companies

```

# Fetch companies
companies_endpoint <- "/crm/v3/objects/companies"
company_properties <- c("name", "domain", "industry", "city", "state", "country")
companies_url <- paste0(companies_endpoint, "?properties=", paste(company_properties, collapse=", "))

companies_response <- hubspot_request(companies_url)

# Convert to data frame
companies_df <- map_dfr(companies_response$results, function(company) {
  props <- company$properties
  tibble(
    id = company$id,
    name = props$name %||% NA,
    domain = props$domain %||% NA,
    industry = props$industry %||% NA,
    city = props$city %||% NA,
    state = props$state %||% NA,
    country = props$country %||% NA
  )
})

kable(head(companies_df), caption = "Sample HubSpot Companies")

```

Table 2: Sample HubSpot Companies

id	name	domain	industry	city	state	country
3369997040	TM Mobile	t-mobile.com	TELECOMMUNICATIONS	Bellevue	WA	United States
3889131916	rstudio	rstudio.com	EDUCATION_MANAGEMENT	Boston	MA	United States
3891907013	Genentech	gene.com	PHARMACEUTICALS	South San Francisco	California	United States
3897672158	Bill & Melinda Gates Foundation	gatesfoundation.org	HEALTH_WELLNESS_AND_FITNESS	Seattle	WA	United States
4072587498	Procogia	procogia.com	INFORMATION_TECHNOLOGY_AND_SERVICES	San Jose	CA	United States
4081344564	Roche	roche.com	PHARMACEUTICALS	Basel	Basel-Stadt	Switzerland

Working with Deals

```
# Function to get total count of deals without retrieving all data
get_deals_count <- function() {
  tryCatch({
    # Use the search endpoint to get just the count
    search_endpoint <- "/crm/v3/objects/deals/search"

    search_body <- list(
      filterGroups = list(), # No filters = all deals
      sorts = list(),
      properties = list("hs_object_id"), # Minimal property to reduce response size
      limit = 1, # We only need the count, not the actual data
      after = 0
    )

    response <- hubspot_request(search_endpoint, method = "POST", body = search_body)

    # Return the total count
    return(response$total)

  }, error = function(e) {
    cat("Error getting deals count:", e$message, "\n")
    return(NULL)
  })
}

# Get the actual total count
total_deals_count <- get_deals_count()
```

Fetch Deals

```
# Fetch deals
deals_endpoint <- "/crm/v3/objects/deals"
deal_properties <- c("dealname", "amount", "dealstage", "pipeline", "createdate", "closedate")
deals_url <- paste0(deals_endpoint, "?properties=", paste(deal_properties, collapse = ","))

deals_response <- hubspot_request(deals_url)

# Convert to data frame
```



```

deals_df <- map_dfr(deals_response$results, function(deal) {
  props <- deal$properties
  tibble(
    id = deal$id,
    dealname = props$dealname %||% NA,
    amount = as.numeric(props$amount %||% 0),
    dealstage = props$dealstage %||% NA,
    pipeline = props$pipeline %||% NA,
    createdate = props$createdate %||% NA,
    closedate = props$closedate %||% NA
  )
})

kable(deals_df, caption = "Sample HubSpot Deals")

```

Table 3: Sample HubSpot Deals

id	dealname	amount	dealstage	pipeline	createdate	closedate
1994350515	Mobile - Eric Lee - Opportunity	50000	closedlost	default	2020-05-14T20:34:09.345Z	2020-05-22T16:38:52.749Z
1994392986	Gutam Bharti, T-mobile (Gary & Zilai June - December)	210000	open	default	2020-05-14T20:39:51.085Z	2020-08-07T19:34:19.235Z
1994394679	San Francisco Ruis, Microsoft -Data Engineer	200000	closedlost	default	2020-05-14T20:29:08.091Z	2021-01-19T20:22:08.705Z
1994801076	Mobile (Derrick Ives) P&T - Akbar	150000	open	default	2020-05-14T20:37:47.255Z	2020-08-07T19:27:34.869Z
2017631183	Aren Juhl, Microsoft, Insights Project	120000	closedlost	default	2020-05-21T19:04:56.256Z	2020-09-15T23:52:35.819Z
2017637512	Steven Clanton, NCCI , RStudio project	50000	closedlost	default	2020-05-21T19:08:36.008Z	2020-11-20T01:28:17.256Z
2018557902	Party Hall, B&M Gates, RStudio connect migration	40000	closedlost	default	2020-05-22T00:18:36.893Z	2020-12-03T04:13:41.406Z
2020940779	Translink RFP - Various Resources - April Submission	100000	closedlost	default	2020-05-22T16:32:33.044Z	2020-09-15T23:52:27.259Z
2237438722	Project Manager role	112000	closedlost	default	2020-05-27T21:47:05.448Z	2020-07-21T18:21:03.827Z
2246738616	Merry - Intros to business team	10000	closedlost	default	2020-06-01T21:35:07.545Z	2020-12-03T19:42:37.554Z

Deal Analytics

```
if (nrow(deals_df) > 0) {
  # Get total count of all deals in HubSpot
  total_deals_count <- get_deals_count()

  # Basic deal statistics
  cat(" Deal Summary Statistics\n")
  cat("=====\n")

  if (!is.null(total_deals_count)) {
    cat("Total deals in HubSpot:", total_deals_count, "\n")
    cat("Deals retrieved:", nrow(deals_df), "\n")
    cat("Sample percentage:", round(nrow(deals_df) / total_deals_count * 100, 1), "%\n")
  } else {
    cat("Deals retrieved:", nrow(deals_df), "(unable to get total count)\n")
  }

  cat("Total deal value (sample):", scales::dollar(sum(deals_df$amount, na.rm = TRUE)), "\n")
  cat("Average deal value (sample):", scales::dollar(mean(deals_df$amount, na.rm = TRUE)), "\n")

  # Deal value distribution
  if (sum(!is.na(deals_df$amount)) > 0) {
    plot_title <- if (!is.null(total_deals_count)) {
      paste0("Distribution of Deal Values (Sample of ", nrow(deals_df), " from ", total_deals_count, " deals)")
    } else {
      paste0("Distribution of Deal Values (Sample of ", nrow(deals_df), " deals)")
    }

    ggplot(deals_df |> filter(!is.na(amount), amount > 0), aes(x = amount)) +
      geom_histogram(bins = 20, fill = "steelblue", alpha = 0.7) +
      labs(
        title = "Distribution of Deal Values",
        subtitle = plot_title,
        x = "Deal Amount ($)",
        y = "Count"
      ) +
      scale_x_continuous(labels = scales::dollar_format()) +
      theme_minimal()
  }
}
```

Deal Summary Statistics

=====

Total deals in HubSpot: 621

Deals retrieved: 10

Sample percentage: 1.6 %

Total deal value (sample): \$1,042,000

Average deal value (sample): \$104,200

Distribution of Deal Values

Distribution of Deal Values (Sample of 10 from 621 total deals)



Creating a Contact (Example)

Here's how you would create a new contact:

```
# Example of creating a new contact (set eval=FALSE to prevent accidental execution)
new_contact_data <- list(
  properties = list(
    firstname = "John",
    lastname = "Doe",
    email = "john.doe@example.com",
    company = "Example Corp"
  )
)
```

```
# Create the contact
create_response <- hubspot_request(
  "/crm/v3/objects/contacts",
  method = "POST",
  body = new_contact_data
)

cat("Created contact with ID:", create_response$id, "\n")
```

Note - I've disabled write permissions for this API for now – DAH

Best Practices and Tips

Rate Limiting

HubSpot has rate limits. For production applications: - Implement retry logic with exponential backoff - Monitor your API usage - Use batch operations when available

Error Handling

Always wrap API calls in proper error handling:

```
safe_hubspot_request <- function(endpoint) {
  tryCatch({
    hubspot_request(endpoint)
  }, error = function(e) {
    cat("API Error:", e$message, "\n")
    return(NULL)
  })
}
```

Data Processing

Consider using `purrr` functions for cleaner data processing:

```
# Example of processing multiple objects
process_contacts <- function(contacts_list) {
  contacts_list |>
  map_dfr(~ {
```

```

    tibble(
      id = .x$id,
      email = .x$properties$email %||% NA,
      firstname = .x$properties$firstname %||% NA,
      lastname = .x$properties$lastname %||% NA
    )
  })
}

```

Summary

This document demonstrated how to:

1. Authenticate with HubSpot API using access tokens
2. Create helper functions for API interactions
3. Fetch and process contacts, companies, and deals
4. Handle pagination for large datasets
5. Create new records (contacts example)
6. Implement basic analytics and visualization

For production use, consider: - Implementing comprehensive error handling - Adding rate limit management - Using HubSpot's batch endpoints for bulk operations - Storing credentials securely using services like AWS Secrets Manager or similar

Resources

- [HubSpot API Documentation](#)
- [HubSpot Private Apps Guide](#)
- [httr2 Package Documentation](#)